

ClaimFlow AI — Technical Specification

Architecture, workflow and data-processing mechanics of the ClaimFlow AI insurance-claims platform

Document type	Technical specification (as-built)
System	ClaimFlow AI — health-insurance claims intake, AI-assisted triage, human review and settlement
Code baseline	<code>main</code> @ <code>f6ef61d</code> (2026-10-05)
Source of truth	This document describes what the code does. Where <code>SPEC.md</code> and the code differ, the code is described and the difference is noted.

How to read this document. It follows a data-platform template (ingestion → raw storage → transformation → reconciliation → serving → orchestration). ClaimFlow is a transactional claims-processing system, not a batch analytics platform, so each section maps the template's concept onto what ClaimFlow actually does: *ingestion* is claim intake from three channels, *transformation* is AI evidence extraction and enrichment, and *reconciliation* is the validation-and-matching engine that reconciles each claim against policy, coverage, claimant identity, prior claims and the claim's own documents before routing it to a human. No component is described that does not exist in the code.

Table of contents

- 1. System Overview & Architecture Principles
- 2. End-to-End Data Pipeline (Step-by-Step)
 - Step 1 — Ingestion
 - Step 2 — Storage & Staging (Raw Layer)
 - Step 3 — Transformation & Processing Engine
 - Step 4 — Validation, Matching & Routing Engine (the "reconciliation" layer)
 - Step 5 — Serving & Consumption Layer
- 3. Component-to-Technology Mapping Matrix
- 4. Orchestration, Monitoring & Error Handling
- 5. Appendix A — Data Model
- 6. Appendix B — Process Variables Contract
- 7. Appendix C — Runtime Topology & Configuration
- 8. Appendix D — Known Technical Limitations (as-built)

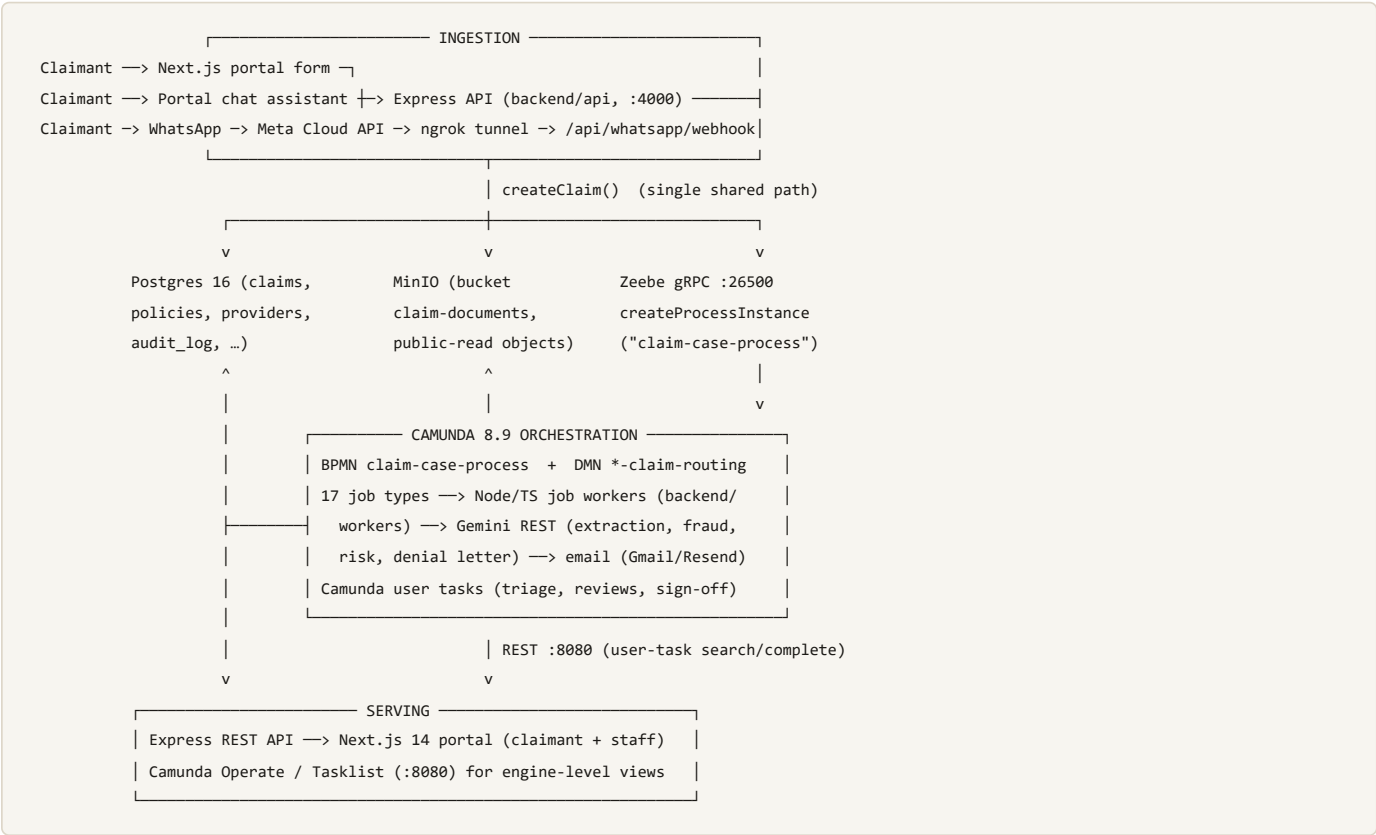
1. System Overview & Architecture Principles

1.1 What the system does

ClaimFlow AI takes a health-insurance claim from first notice of loss (FNOL) to a closed case:

- 1. A claimant files a claim — through the web portal form, the portal's chat assistant, or a WhatsApp bot — with supporting documents (bills, discharge summaries, prescriptions).
- 2. The claim is persisted to Postgres, its documents to MinIO object storage, and a **Camunda 8 (Zeebe)** BPMN process instance is started for it.
- 3. Deterministic validation reconciles the claim against the policy book: active policy on the incident date, authorized claimant, no open duplicate claim, required fields present.
- 4. **Google Gemini** reads every attached document, extracts structured evidence, writes a case summary, flags fraud indicators, and produces a 0–100 risk score with reasoning.
- 5. A **DMN decision table** suggests which reviewer role should handle the claim (adjuster, investigator, legal). A **human triage reviewer always confirms or overrides** that suggestion.
- 6. A **human reviewer always makes the decision**: approve, deny, or request more information (which loops back to the claimant and then to the same reviewer).
- 7. Approved claims over \$50,000 need a second human sign-off by a supervisor. Approved claims trigger a (mock) settlement; denied claims get an AI-drafted denial letter. The claimant is notified by email and the case is closed.
- 8. Every automated and human step writes a row to `audit_log`, the durable, queryable case history.

1.2 Architecture at a glance



1.3 Core design patterns

Pattern	How it is applied
Process orchestration (BPMN) over choreography	One BPMN process instance per claim (<code>claim-case-process</code>) owns the end-to-end flow. Services don't call each other; Zeebe activates jobs and workers complete them. Camunda 8 has no CMMN engine, so the "case" is the process instance plus <code>audit_log</code> .
External task / job-worker pattern	Every automated step is a BPMN service task with a <code>zeebe:taskDefinition</code> type . Stateless Node.js workers long-poll Zeebe over gRPC, do one unit of work, write to Postgres, and return output variables.
Event-driven, per-record processing (not batch)	Each claim is processed individually as it arrives. There are no scheduled batch jobs, no micro-batches and no streaming platform. Time-based behavior (SLAs) is modeled as BPMN timer boundary events.
Decision-as-data (DMN)	Reviewer routing is a FIRST-hit DMN table evaluated by the engine, selected dynamically per insurance type (<code>=insuranceType + "-claim-routing-decision"</code>).
Human-in-the-loop by construction	AI output is advisory. Triage confirmation and the approve/deny decision are mandatory Camunda user tasks; there is no straight-through approval path.
System of record vs. engine state	Postgres is the business system of record (claim columns, documents, fraud indicators, audit trail). Zeebe holds only in-flight process state and variables.
Append-only case history	<code>audit_log</code> gets at least one row per worker execution and per user-task completion, with <code>actor_type</code> <code>∈ { system , ai , human }</code> .
Single write path for intake	All three channels converge on one function, <code>createClaim()</code> (<code>backend/api/src/create-claim.ts</code>), so validation, persistence and process start are identical regardless of channel.
Strategy/provider interfaces	<code>SettlementProvider</code> (mock only), <code>NotificationProvider</code> (Gmail → Resend → console mock, chosen at startup), and an insurance-type registry (<code>backend/shared/insurance-types/</code>) for per-line-of-business prompts and required fields.
Model fallback chain	Gemini calls walk an ordered list of models on overload/quota errors (HTTP 429/5xx).

2. End-to-End Data Pipeline (Step-by-Step)

Step 1 — Ingestion

1.1 Source types and entry points

#	Source	Entry point	Protocol / format	Trigger
1	Portal claim form (/claims/new , components/ClaimForm.tsx)	POST /api/claims	multipart/form-data (fields + documents[] files), Bearer JWT	User submits the 5-step form
2	Portal chat assistant (components/assistant/AssistantChat.tsx)	POST /api/claims with extra field source=chat	multipart/form-data, Bearer JWT	User confirms the chat's review card
3	WhatsApp bot	POST /api/whatsapp/webhook (Meta Cloud API → ngrok static domain → API)	JSON webhook events, HMAC-SHA256 signed (X-Hub-Signature-256); media pulled from Graph API v26.0	Each inbound WhatsApp message
4	Claimant resubmission (after a "more info" request)	POST /api/claims/:id/resubmit	multipart/form-data (documents[] , note), Bearer JWT	Claimant uploads requested documents
5	Policy book (reference data)	POST /api/policies	JSON, Bearer JWT, admin role	Admin adds a policy (+ dependents) in the portal
6	Provider reference data	Find-or-create inside createClaim() by NPI	—	Any claim with a new NPI

Ingestion frequency: **on demand, per event**. There are no file drops, SFTP feeds, CDC streams or scheduled loads.

1.2 Channel mechanics

Portal form and chat (sources 1–2)

- Express 4 + **multer 2** with memoryStorage , field name documents , **10 MB per-file limit** (routes/claims.ts). Oversize uploads return HTTP 400.
- Accepted types are restricted client-side to PDF/JPG/PNG (accept=".pdf,.jpg,.jpeg,.png"); the server does not filter MIME types.
- Fields: policyNumber, claimType, claimantName, claimantEmail, claimantPhone, channel, source, incidentDate, incidentDescription, claimAmount, diagnosisCode, procedureCode, providerNpi, providerTaxId, facilityName, facilityAddress, serviceDateFrom, serviceDateTo, totalBilledAmount, coordinationOfBenefits, attested .
- The chat assistant collects the same fields one question at a time, using the same portal widgets (ICD-10 lookup against the NLM Clinical Tables API, provider search, date pickers), and holds its draft in the tab's sessionStorage until submit. It adds source=chat , which only changes the audit row's detail.source to claimant-portal-chat .

WhatsApp (source 3) — backend/api/src/routes/whatsapp.ts

- GET /webhook answers Meta's verify handshake (hub.mode=subscribe + WHATSAPP_WEBHOOK_VERIFY_TOKEN).
- POST /webhook : HMAC-SHA256 over the **raw request body** (captured by the JSON parser's verify hook) keyed with WHATSAPP_APP_SECRET , compared with timingSafeEqual . Invalid → 401. Valid → **HTTP 200 is returned before processing** (Meta retries slow acks).
- Idempotency:** INSERT INTO whatsapp_processed_messages (message_id) ... ON CONFLICT DO NOTHING ; a redelivered message id is dropped.
- Sender resolution:** the phone must match policies.policyholder_phone , policy_dependents.phone or an existing claims.claimant_phone ; otherwise a fixed "not linked" reply is sent and no session is created.
- Conversation state:** one whatsapp_sessions row per phone (mode ∈ menu | claim_status | policy_status | raising_claim , collected_fields jsonb , documents jsonb).
- Raise-a-claim:** 18 sequential steps (policy list, claim-type list, yes/no buttons, flexible date parsing) with per-step validation using the same regexes as the API; then documents. Each image/document message is downloaded from the Graph API and written to MinIO immediately (\${Date.now()}-\${filename}), and its URL appended to the session.
- done → raiseClaim() → createClaim({...fields, channel: "whatsapp", claimantPhone}) . A ClaimValidationError carrying a field re-asks only that question.

Resubmission (source 4) — claimant-only, ownership by email, requires status = 'awaiting_info' . Finds the open Task_ClaimantProvideMoreInfo user task by process instance key via the Camunda REST API, inserts the new documents and an audit row in one transaction, then completes the user task with {claimId, resubmittedByUserId, documentCount} .

1.3 The single intake transaction — createClaim()

Every claim from every channel goes through this sequence:

Order	Operation	Detail
1	Schema/format validation	Required fields; attested = true ; ICD-10 <code>^[A-TV-Z][0-9][0-9AB](\.[0-9A-Z]{1,4})?;\$</code> ; CPT/HCPCS <code>^(\d{5} [A-Z]\d{4})\$</code> ; NPI <code>^[0-9]{10}\$</code> ; total billed > 0 ; ≥ 1 document. Errors are <code>ClaimValidationError(message, field)</code> .
2	BEGIN	One Postgres transaction for all writes below.
3	Policy lookup	<code>SELECT ... FROM policies WHERE policy_number = \$1 → 400 if none.</code>
4	Coverage limit	<code>claimAmount <= policies.coverage_amount → 400 (field: claimAmount) otherwise.</code>
5	Provider upsert	Find by NPI; reuse unchanged if found, else <code>INSERT (NPI, tax ID, facility name/address).</code>
6	Claim insert	<code>status = 'submitted' , attestation_signed_at = now() , service_date_to defaults to service_date_from , carrier_id / insurance_type copied from the policy, channel , claimant_phone .</code>
7	Documents	For each file: <code>putObject</code> to MinIO (portal uploads; WhatsApp files are already there), <code>INSERT claim_documents (claim_id, file_url)</code> ; images ≤ 2 MB also get a <code>base64 dataUri</code> for the process variables.
8	Audit	<code>audit_log (system, backend/api, 'submitted', {source, documentCount}) .</code>
9	COMMIT	
10	Process start (after commit)	Zeebe gRPC <code>createProcessInstance('claim-case-process', variables)</code> — see Appendix B.
11	Link	<code>UPDATE claims SET process_instance_key , audit process-started .</code>
12	Duplicate-instance guard	Searches process variables for the same <code>claimId</code> ; cancels any other instance and audits <code>duplicate-process-cancelled</code> .

If Zeebe is unreachable at step 10, the error is logged and the claim row remains with a null `process_instance_key` (see Appendix D).

Step 2 — Storage & Staging (Raw Layer)

ClaimFlow has no lakehouse or file-based raw zone. The "raw layer" is the combination of the **document object store** (binary originals) and the **intake columns of the `claims` row** (which no later step overwrites).

2.1 Storage services

Store	Technology	What it holds	Format
Object storage	MinIO (S3-compatible), image <code>quay.io/minio/minio</code> , ports 9000 (S3 API) / 9001 (console)	Original claim documents from all channels	Binary objects as uploaded (PDF, JPEG, PNG). Bucket <code>claim-documents</code> (env <code>MINIO_BUCKET</code>), created on API startup with an anonymous <code>s3:GetObject</code> bucket policy.
Relational system of record	PostgreSQL 16 (<code>postgres:16-alpine</code>), port 5432	Claims, policies, dependents, providers, document references and AI extractions, fraud indicators, audit trail, users, WhatsApp sessions	Row store; <code>jsonb</code> for semi-structured data (<code>claim_documents.extracted_data</code> , <code>audit_log.detail</code> , <code>whatsapp_sessions.collected_fields / documents</code>)
Process state	Camunda 8.9.16 Zeebe broker; secondary storage H2 file DB (<code>jdbc:h2:file:./camunda-data/h2db</code>) via the <code>rdbs</code> exporter	In-flight process instances, variables, user tasks, incidents; history for Operate/Tasklist	Zeebe internal log + H2
Browser session storage	<code>window.sessionStorage</code> (per tab)	Bearer token + user JSON; chat-assistant draft	JSON strings

2.2 Object layout and addressing

- Key: `$(Date.now())-$(originalFilename)` — flat namespace, no prefix partitioning.
- Reference stored in Postgres: full public URL `http://{MINIO_ENDPOINT}:{MINIO_PORT}/claim-documents/{key}` in `claim_documents.file_url` .
- Consumers (workers, portal `<img>` / `<iframe>` previews) read objects directly by that URL; Gemini receives them as `base64 inlineData` fetched by the worker.

2.3 Partitioning, indexing and retention

- **Partitioning:** none (no table partitioning, no object-key partitioning). Volumes are claim-scale, not event-scale.
- **Indexes:** `claims(carrier_id)` , `claims(status)` , `claims(process_instance_key)` , `claims(policy_id)` , `claims(provider_id)` ; `policies(carrier_id)` ; `claim_documents(claim_id)` ; `audit_log(claim_id)` , `audit_log(created_at)` ; unique `users(lower(email))` ; unique `policies(policy_number)` , `providers(npi)` , `policy_dependents(policy_id, email)` .

- **Multi-carrier field:** `carrier_id` (uuid) is present and indexed on `claims` and `policies` and participates in policy matching; isolation by carrier is not enforced at the API.
- **Retention:** no TTL or archival jobs. `whatsapp_sessions` and `whatsapp_processed_messages` grow without cleanup.

2.4 Schema management

Forward-only SQL migrations (`backend/db/migrations/0001...0018` , no `0004`) applied by `backend/db/run-migrations.sh` : each file runs once inside `psql --single-transaction -v ON_ERROR_STOP=1` in the Postgres container, and its version is recorded in `schema_migrations` . No ORM.

Step 3 — Transformation & Processing Engine

Processing is performed by **Node.js/TypeScript job workers** (`backend/workers/` , 17 job types) activated by Zeebe, plus the **Gemini** generative model for document understanding. There is no Spark, Pandas or SQL-warehouse compute; transformations are per-claim and executed inside workers and Postgres statements.

3.1 Execution engine

Aspect	Implementation
Runtime	Node.js (v24 locally), TypeScript 5.5, run with <code>tsx</code>
Engine client	<code>@camunda8/sdk ^8.8.13</code> — <code>ZeebeGrpcClient</code> , <code>createWorker({ taskType, taskHandler })</code> per job type
Concurrency	SDK defaults (no custom <code>maxJobsToActivate</code> , <code>timeout</code> or poll interval)
State	Workers are stateless; all state is in Zeebe variables and Postgres
AI	<code>backend/shared/gemini-client.ts</code> : <code>raw fetch</code> to <code>generativelanguage.googleapis.com/v1beta/models/{model}:generateContent</code> , <code>temperature: 0</code> , JSON responses parsed after stripping <code>```json</code> fences
Model selection	<code>GEMINI_MODEL</code> (code default <code>gemini-flash-latest</code> ; local config <code>gemini-3.6-flash</code>) then <code>GEMINI_FALLBACK_MODELS</code> in order (local: <code>gemini-3.5-flash</code> , <code>gemini-3.5-flash-lite</code> , <code>gemini-3.1-flash-lite</code>). HTTP 429/500/502/503/504 → next model; any other error or an empty response throws.
Per-line-of-business config	<code>getInsuranceTypeConfig(insuranceType)</code> → <code>{ requiredFields, documentTypes, promptTemplate, fraudPromptTemplate }</code> ; only <code>health</code> is registered

3.2 Transformation steps (in process order)

Step	Job type	Actor	Input	Logic	Writes	Output variables
Validation & enrichment	<code>validate-claim</code>	system	<code>claimId</code>	See Step 4.1 (reconciliation). Also derives two routing features.	<code>claims.policy_id</code> ; <code>status='validating'</code> on pass	<code>validationPassed</code> , <code>policyId</code> , <code>duplicatePendingClaim</code> , <code>duplicateClaimId</code> , <code>authorizedClaimant</code> , <code>daysSincePolicyEffective</code> , <code>claimantClaimCountLast12Months</code> , <code>slaDeadline</code>
Evidence extraction	<code>extract-evidence</code>	ai	<code>claimId</code> , <code>documents[]</code>	One Gemini call per claim with all documents attached as parts plus the incident narrative; prompt <code>health.promptTemplate</code> (version <code>v2-narrative-cross-check</code>). Expected JSON: <code>{ caseSummary, documents: [{ documentIndex, extractedData }] }</code> . Zero documents → fixed summary, no AI call.	<code>claims.case_summary</code> ; <code>claim_documents.extracted_data</code> (free-form jsonb per document)	<code>caseSummary</code>

Step	Job type	Actor	Input	Logic	Writes	Output variables
Fraud-indicator detection	detect-fraud-indicators	ai	case summary, extracted data, claimant name, narrative	Gemini (v4-narrative-mismatch-check) classifies indicators into claimant-identity mismatch, cross-document inconsistency, coding/billing, narrative mismatch, missing/placeholder data, other; confidence bands 0.9–1.0 / 0.6–0.8 / 0.3–0.5.	Replaces <code>claim_fraud_indicators</code> rows; <code>claims.fraud_indicator_count</code> = count with confidence ≥ 0.5	<code>fraudIndicatorCount</code>
Risk scoring	score-risk	ai	<code>claimAmount</code> , <code>fraudIndicatorCount</code> , <code>caseSummary</code>	Gemini rubric (v2-rubric) → integer 0–100 in bands 0–19 / 20–39 / 40–59 / 60–79 / 80–100, plus reasoning.	<code>claims.risk_score</code> , <code>claims.risk_reasoning</code>	<code>riskScore</code> , <code>riskReasoning</code>
Denial letter	draft-denial-letter	ai	decision context, denial reason	Gemini drafts a plain-text letter (v1).	<code>claims.denial_letter_text</code>	<code>denialLetterText</code>

3.3 Standardization and normalization rules

Concern	Rule	Where
Medical codes	ICD-10-CM and CPT/HCPCS validated by regex; uppercased on submit	Client (<code>ClaimForm</code> , chat, WhatsApp) and server (<code>createClaim</code>)
Provider identity	NPI is the natural key (10 digits, unique); first-seen facility data is kept	<code>createClaim</code> provider upsert
Amounts	<code>numeric</code> in Postgres; USD only; no currency conversion exists	All layers
Dates	ISO <code>YYYY-MM-DD</code> for clinical dates (<code>date</code>), <code>timestampz</code> (UTC) for events; WhatsApp accepts <code>YYYY-MM-DD</code> , <code>DD/MM/YYYY</code> , <code>DD-MM-YYYY</code> , <code>3 Oct 2026</code> , <code>today</code> , <code>yesterday</code> and normalizes to ISO	<code>createClaim</code> , WhatsApp parser
Service period	<code>service_date_to</code> defaults to <code>service_date_from</code> ; CHECK (to $\geq$ from)	DB constraint
Email identity	Compared case-insensitively (<code>lower(...)</code>) everywhere	API scoping, validation
Phone identity	Digits only with country code, no <code>+</code> (WhatsApp <code>from</code> format), exact match	WhatsApp, <code>validate-claim</code>
Display IDs	<code>#</code> + first 8 hex chars of the claim UUID	<code>shortClaimId</code> (frontend + backend copies)

3.4 Aggregations

Aggregations are computed on read, not materialized:

- `claimantClaimCountLast12Months` — `COUNT(*)` of the claimant's other claims (case-insensitive email) with `created_at` $\geq$ `now()` - 12 months (DMN feature).
- `daysSincePolicyEffective` — `incident_date` - `effective_date` (DMN feature).
- Portal KPIs (total, active, total claimed value, needs-attention) — computed client-side from `GET /api/claims` .

Step 4 — Validation, Matching & Routing Engine (the "reconciliation" layer)

In a reconciliation platform, this layer matches records across sources and raises breaks. ClaimFlow's equivalent reconciles **each claim against four independent sources of truth** and raises a **validation exception** (the "break") for a human when they disagree:

Source A (claim)	Reconciled against (source B)	Match rule
<code>policy_number</code> , <code>carrier_id</code> , <code>incident_date</code>	Policy book (<code>policies</code>)	Exact policy number + carrier, <code>status = 'active'</code> , incident date within <code>[effective_date, expiry_date]</code>
<code>claim_amount</code>	Policy coverage	<code>claim_amount</code> $\leq$ <code>coverage_amount</code> (enforced at intake)
Claimant identity (email/name, or phone for WhatsApp)	Policyholder + <code>policy_dependents</code>	Portal: <code>lower(email)</code> or <code>lower(name)</code> matches policyholder or a dependent. WhatsApp: trimmed phone equals <code>policyholder_phone</code> or a dependent's phone.

Source A (claim)	Reconciled against (source B)	Match rule
The claim itself	Open claims on the same policy	Duplicate = any <i>other</i> claim on the same <code>policy_id</code> with <code>status NOT IN ('approved', 'denied')</code> (no time window)
Narrative and form fields	The claim's own documents	AI cross-check in <code>extract-evidence</code> / <code>detect-fraud-indicators</code> (identity, dates, codes, amounts, narrative consistency)

4.1 Deterministic matching — `validate-claim`

- 1. Load the claim row by `claimId`.
- 2. **Completeness:** every column in `health.requiredFields` (`claimant_name`, `claimant_email`, `incident_date`, `incident_description`, `claim_amount`, `diagnosis_code`, `procedure_code`, `service_date_from`, `provider_id`) must be present → `missingFields[]`.
- 3. **Policy match:** `SELECT ... FROM policies WHERE policy_number = $1 AND carrier_id = $2 AND status = 'active' AND incident_date BETWEEN effective_date AND expiry_date`.
- 4. **Duplicate detection:** open claim on the same policy (rule above) → `duplicatePendingClaim`, `duplicateClaimId`.
- 5. **Authorized claimant:** channel-specific rule above (defaults to true when no policy matched, since step 3 already fails).
- 6. **Verdict:** `validationPassed = missingFields.length === 0 && policyMatched && !duplicate && authorizedClaimant`.
- 7. Persist `policy_id` (null when unmatched); on pass set `status = 'validating'`.
- 8. Derive DMN features `daysSincePolicyEffective`, `claimantClaimCountLast12Months`.
- 9. On fail: email the `triage-team` role and compute `slaDeadline` (24 business hours — see 4.5).
- 10. Audit `validated` with all check results.

4.2 Break handling — Validation Exception Review

When `validationPassed = false` the BPMN gateway routes to the **Validation Exception Review** user task (candidate group `triage-team`, form `ValidationExceptionReviewForm`):

Outcome	Mechanism	Effect
Resolve (override)	Reviewer sets <code>resolutionAction = "resolve"</code> → <code>capture-validation-exception</code>	<code>status = 'validating'</code> ; audit <code>validation_exception_resolved</code> (with <code>overrideNoPolicyMatch</code> when no policy was matched); claim continues to AI extraction
Reject	<code>resolutionAction = "reject" + denialReason</code>	<code>decision = 'deny'</code> , <code>status = 'denied'</code> ; audit <code>validation_exception_rejected</code> ; → denial letter
SLA expiry	Interrupting timer boundary event at <code>slaDeadline</code> → <code>auto-reject-validation-exception</code>	Fixed reason "Auto-rejected: validation exception unresolved after 24 business hours"; → denial letter

4.3 Probabilistic matching — AI cross-document checks

`extract-evidence` and `detect-fraud-indicators` compare the claim's declared facts (name, dates, codes, amounts, narrative) with what the documents actually say. Mismatches become rows in `claim_fraud_indicators` (`type`, `description`, `confidence`). Only indicators with confidence ≥ 0.5 count toward `fraud_indicator_count`, which drives routing. These are **advisory**: they never deny a claim by themselves.

4.4 Routing decision — DMN `health-claim-routing-decision`

Business rule task `Task_RoutingDecision`, `decisionId = insuranceType + "-claim-routing-decision"`, result variable `assignedRole`, hit policy **FIRST**:

#	Condition	<code>assignedRole</code>
1	<code>claimAmount > 50000</code>	<code>legal</code>
2	<code>fraudIndicatorCount >= 1</code>	<code>investigator</code>
3	<code>daysSincePolicyEffective <= 14</code>	<code>investigator</code>
4	<code>claimantClaimCountLast12Months >= 3</code>	<code>investigator</code>
5	<code>riskScore >= 40</code>	<code>adjuster</code>
6	<code>claimAmount > 5000</code>	<code>adjuster</code>
7	(otherwise)	<code>adjuster</code>

`capture-routing-decision` persists `assigned_role`, sets `status = 'triage'`, emails the triage team and sets a new `slaDeadline`.

4.5 Human confirmation, decision and escalation

Stage	User task (candidate group)	Worker on completion	Possible outcomes
Triage	Task_TriageReview (triage-team)	capture-triage-review	Confirm/override confirmedRole (adjuster/investigator/legal; override flagged in audit) → status = 'in_review' ; or reject with reason → denial
Role review	Task_AdjusterReview (adjusters), Task_InvestigatorReview (investigators), Task_LegalReview (legal-reviewers), Task_SupervisorReview (supervisors)	capture-review-decision	approve → approved ; deny (reason required) → denied ; moreInfo (reason required) → awaiting_info
More info	Task_ClaimantProvideMoreInfo (no group; completed through POST /api/claims/:id/resubmit)	capture-claimant-resubmission	status = 'in_review' ; routes back to the same reviewing role
Second sign-off	Task_SupervisorSignoff (supervisors) when approved and claimAmount > 50000	capture-signoff	Audit only, then settlement

SLA timers. Every human queue has an interrupting timer boundary event whose `timeDate` is the FEEL expression `date and time(slaDeadline)` . `slaDeadline` is computed by workers as **now + 24 business hours**, skipping weekends and US federal holidays (`business-days.ts`). On expiry:

Queue	Timeout behavior
Validation exception	auto-reject-validation-exception → denial
Triage	auto-confirm-triage — accepts the DMN suggestion as <code>confirmedRole</code>
Adjuster review	auto-escalate-review → investigator
Investigator review	auto-escalate-review → legal
Legal review	auto-escalate-review → supervisor
Supervisor review	No timer

4.6 Settlement and closure

Path	Workers (in order)	Writes
Approved	trigger-settlement → notify-claimant → close-case	settlement_id (mock <code>mock-settlement-<uuid></code>); email; status = 'approved'
Denied	draft-denial-letter → notify-claimant → close-case	denial_letter_text ; email with the letter; status = 'denied'

Step 5 — Serving & Consumption Layer

5.1 Serving API — Express 4 (backend/api , port 4000)

Middleware order: CORS (origin `CORS_ORIGIN` , allowed headers `Content-Type` , `Authorization` , no credentials) → JSON body parser (raw body retained for WhatsApp signatures) → `attachUser` (verifies the HS256 Bearer JWT and its `token_version` against `users`) → routers.

Router	Endpoints	Access
/api/auth	POST /signup , POST /login → { access_token , token_type: "bearer" , user } ; GET /me ; POST /logout ; POST /forgot-password , POST /verify-otp (bumps <code>token_version</code>); POST /register-staff	Public / authenticated / admin
/api/claims	GET / (claimants scoped to own email), GET /:id (+ documents, fraud indicators, last reviewer action), GET /:id/audit-log (filters <code>actorType</code> , <code>from</code> , <code>to</code>), GET /:id/pending-task , POST /:id/resubmit , POST /	Authenticated; audit log staff-only
/api/policies	GET / , GET /:id (+ dependents), POST / , DELETE /:id	Authenticated; write admin-only
/api/providers	GET /	Open
/api/tasks	GET / , GET /:key , POST /:key/claim , POST /:key/unclaim , POST /:key/complete — proxies Camunda's REST user-task API, filtered by the caller's role → candidate group (<code>adjuster</code> → <code>adjusters</code> , <code>investigator</code> → <code>investigators</code> , <code>legal-reviewer</code> → <code>legal-reviewers</code> , <code>supervisor</code> → <code>supervisors</code> , <code>triage-team</code> → <code>triage-team</code> ; admin unfiltered)	Staff roles
/api/assistant	GET /claims , GET /claims/:id — shared status wording with the WhatsApp bot	Claimant
/api/whatsapp	GET /webhook , POST /webhook	Signature-verified

5.2 Presentation — Next.js 14 portal (frontend/portal , port 3000)

Route	Audience	Content
/	All	Claims grid with KPIs (claimants: own claims)
/claims/new	Claimant	5-step claim form
/claims/[id]	Owner / staff	Stage tracker, amounts, risk, AI case summary and fraud indicators, service & billing, provider, incident, review & decision, documents (inline image/PDF preview), resubmission card
/policies , /policies/[id]	All (claimants: own)	Policy book; admin "add policy" panel
/tasks , /tasks/[key]	Staff	Role-filtered task queue; task detail with case summary, claim details, documents and the decision form
/audit	Staff	audit_log timeline per claim with actor and date filters
/login , /signup , /forgot-password , /admin/register	—	Authentication and staff provisioning
Chat assistant (all claimant pages)	Claimant	Check claim status, check policy status, raise a claim

- **Data access:** a single client (lib/api.ts → apiFetch) sends Authorization: Bearer <token> from the tab's sessionStorage ; each tab can be signed in as a different user. A 401 signs the tab out.
- **Caching:** none. All reads use cache: "no-store" ; there is no CDN, Redis or materialized view. Freshness is per request.
- **Rendering:** client components; the logged-in user is resolved in the browser.

5.3 Other consumers

Consumer	Interface	Purpose
WhatsApp claimants	WhatsApp interactive lists/buttons via Graph API v26.0	Claim status, policy status, raise a claim
Email recipients	Gmail SMTP (nodemailer) or Resend HTTP API	Claimant outcomes; reviewer "new task" notices; OTP codes
Camunda Operate	:8080/operate	Engine-level instance view, variables, incidents
Camunda Tasklist	:8080/tasklist	Alternative task UI (forms deployed with the process)
BI tools	None integrated	Postgres is directly queryable (audit_log is the reporting backbone)

3. Component-to-Technology Mapping Matrix

Phase	Component	Framework / Engine	Storage / Format	Protocol
Ingestion	Portal claim form	Next.js 14.2 / React 18.3 client component	Browser memory → multipart	HTTP POST /api/claims (multipart/form-data, Bearer JWT)
Ingestion	Portal chat assistant	React component + step engine (claim-steps.ts)	sessionStorage draft (JSON)	HTTP multipart, source=chat
Ingestion	WhatsApp channel	Express 4 router, Meta WhatsApp Cloud API v26.0, ngrok static domain	whatsapp_sessions (jsonb), whatsapp_processed_messages	HTTPS webhook (JSON, HMAC-SHA256 signature); Graph API REST for replies and media
Ingestion	File upload handling	multer 2 (memoryStorage , 10 MB/file)	In-memory buffer	multipart
Ingestion	ICD-10 lookup	NLM Clinical Tables API	—	HTTPS GET (browser)
Ingestion	Intake transaction	createClaim() (TypeScript), pg 8	Postgres rows	SQL over TCP 5432
Raw storage	Document store	MinIO (S3-compatible), minio JS client 8	Binary objects (PDF/JPEG/PNG), bucket claim-documents	S3 API over HTTP :9000
Raw storage	System of record	PostgreSQL 16 (alpine)	Relational tables + jsonb	SQL :5432
Raw storage	Schema migrations	run-migrations.sh + psql	schema_migrations	docker-compose exec psql
Orchestration	Process engine	Camunda 8.9.16 (Zeebe broker, single orchestration container)	Zeebe log + H2 file DB (rdbms secondary storage)	gRPC :26500; REST :8080 /v2
Orchestration	Process model	BPMN 2.0 claim-case-process	.bpmn XML (deployed via gRPC)	—
Orchestration	Routing rules	DMN 1.3 health-claim-routing-decision (FIRST)	.dmn XML	Evaluated in-engine

Phase	Component	Framework / Engine	Storage / Format	Protocol
Orchestration	Deployment	<code>backend/deploy-resources.mjs</code> (<code>@camunda8/sdk</code>)	BPMN, DMN, 3 Camunda forms	gRPC
Transformation	Job workers (17)	Node.js + TypeScript (<code>tsx</code>), <code>@camunda8/sdk 8.8 ZeebeGrpcClient</code>	Process variables (JSON); Postgres	gRPC long-poll job activation
Transformation	Document understanding	Google Gemini (<code>generateContent</code> , temperature 0, model fallback chain)	Base64 <code>inlineData</code> in; JSON out → <code>jsonb</code>	HTTPS REST
Reconciliation	Deterministic validation	<code>validate-claim</code> worker (SQL predicates)	Postgres	SQL
Reconciliation	AI cross-checks	<code>extract-evidence</code> , <code>detect-fraud- indicators</code> , <code>score-risk</code> workers + Gemini	<code>claim_fraud_indicators</code> , <code>claims.risk_*</code>	HTTPS REST + SQL
Reconciliation	Routing	Zeebe DMN engine	Process variable <code>assignedRole</code>	In-engine
Reconciliation	Exception & review queues	Camunda user tasks (Zeebe user tasks) + candidate groups	Zeebe state	REST :8080 (search/assign/complete)
Reconciliation	SLA management	BPMN timer boundary events (<code>timeDate</code>), <code>business-days.ts</code>	<code>slaDeadline</code> variable (ISO datetime)	In-engine
Settlement	Payout	<code>SettlementProvider</code> (mock)	<code>claims.settlement_id</code>	In-process
Notification	Email	<code>NotificationProvider</code> : nodemailer (Gmail) / Resend / console mock	—	SMTP / HTTPS
Serving	REST API	Express 4.22, <code>cors</code> , <code>jsonwebtoken</code> (HS256)	Postgres reads	HTTP :4000 JSON
Serving	Web UI	Next.js 14 App Router, React 18	<code>sessionStorage</code> (per-tab token)	HTTP :3000
Serving	Engine UIs	Camunda Operate, Tasklist	Camunda secondary storage	HTTP :8080
Audit	Case history	<code>writeAuditLog()</code> (<code>backend/shared/audit-log.ts</code>)	<code>audit_log</code> (<code>jsonb detail</code>)	SQL
Testing	API tests	Vitest 3 + Supertest 7	Local Postgres (throwaway users)	In-process HTTP
Testing	Portal tests	Vitest 3 + jsdom	—	—

4. Orchestration, Monitoring & Error Handling

4.1 Workflow management

- **Engine:** Camunda 8.9.16 Self-Managed, single `orchestration` container (broker + gateway + Operate + Tasklist + REST API), Docker Compose, `mem_limit: 2560m` , `restart: on-failure` , basic auth `demo/demo` , unprotected REST API, authorizations disabled.
- **Process:** `claim-case-process` — 1 start event, 22 service tasks (17 distinct job types; `notify-claimant` appears 3 times, `auto-escalate-review` 3 times, `close-case` twice), 1 business-rule task, 8 user tasks, 7 exclusive gateways, 5 timer boundary events, 2 end events (`EndEvent_ClaimApproved` , `EndEvent_ClaimDenied`).
- **Instance per claim:** started by `createClaim()` after the DB commit; the `claimId` variable is the business correlation key and is guarded against duplicate instances.
- **Deployment:** `node backend/deploy-resources.mjs` deploys `process/claim-case-process.bpmn` , `process/health-claim-routing.dmn` and three forms (`review-decision` , `triage-review` , `validation-exception-review`). There is no automatic deploy on change.
- **Human work:** staff use the portal's `/tasks` pages (proxying Camunda's REST user-task API) or Camunda Tasklist directly.

Process flow (condensed)

#	BPMN element	Type	Next on the normal path	Branches / timers
1	<code>StartEvent_ClaimSubmitted</code>	Start event	2	—
2	<code>validate-claim</code>	Service task	3	—
3	<code>Gateway_ValidationPassed</code>	Exclusive gateway	5 (pass)	<code>validationPassed = false</code> → 4
4	Validation Exception Review (<code>triage-team</code>) → <code>capture-validation-exception</code>	User task + service task	<code>resolutionAction = "resolve"</code> → 5	Reject → 15. SLA timer → <code>auto-reject-validation-exception</code> → 15

#	BPMN element	Type	Next on the normal path	Branches / timers
5	extract-evidence → detect-fraud-indicators → score-risk	Service tasks (AI)	6	—
6	Task_RoutingDecision (DMN =insuranceType + "-claim-routing-decision") → capture-routing-decision	Business rule task + service task	7	—
7	Triage Review (triage-team) → capture-triage-review	User task + service task	8	triageAction = "reject" → 15. SLA timer → auto-confirm-triage → 8
8	Gateway_RouteByConfirmedRole	Exclusive gateway	Adjuster / Investigator / Legal review (9)	—
9	Adjuster (adjusters), Investigator (investigators), Legal (legal-reviewers), Supervisor (supervisors) review → capture-review-decision	User tasks + service task	10	SLA timers: adjuster → auto-escalate-review → investigator → legal → supervisor (supervisor has no timer)
10	Gateway_Decision	Exclusive gateway	approve → 12	deny → 15; moreInfo → 11
11	notify-claimant → Claimant provides more info → capture-claimant-resubmission → Gateway_RouteBackByConfirmedRole	Service + user task	Back to the same review task (9)	—
12	Gateway_NeedsSecondSignoff	Exclusive gateway	claimAmount <= 50000 → 14	claimAmount > 50000 → 13
13	Supervisor Sign-off (supervisors) → capture-signoff	User task + service task	14	—
14	trigger-settlement → notify-claimant → close-case → EndEvent_ClaimApproved	Service tasks + end event	End	—
15	draft-denial-letter → notify-claimant → close-case → EndEvent_ClaimDenied	Service tasks + end event	End	—

4.2 Retry, incident and failure semantics

Failure	Handling
Worker throws (DB error, Gemini error, validation error)	SDK fails the job; Zeebe retries (default 3 retries) , then raises an incident visible in Operate. No BPMN error boundary events are modeled; resolution is manual (fix cause, resolve incident via Operate or <code>POST /v2/incidents/{key}/resolution</code>).
Gemini overloaded / over quota (429, 5xx)	In-call fallback to the next configured model; only the last model's failure surfaces as a job failure (→ Zeebe retry).
Gemini returns non-JSON / empty	Throws → job retry → incident.
Missing reviewer input (e.g. deny without reason, unknown confirmedRole)	Capture worker throws → retry → incident.
Human inaction	Timer boundary events (Section 4.5) auto-reject, auto-confirm or escalate.
Zeebe unreachable at intake	Error logged; claim persisted without <code>process_instance_key</code> (gRPC client retries up to 5 times first). No outbox or re-drive job.
Duplicate process instance for one claim	Detected after start by searching variables for <code>claimId</code> ; extras cancelled and audited.
WhatsApp redelivery	<code>whatsapp_processed_messages</code> primary key makes processing idempotent.
WhatsApp invalid signature	401, event discarded.
Email provider failure	Reviewer notifications are best-effort (never throw); claimant notification failures throw → job retry.
Engine timeout under memory pressure	Documented: GC pauses can surface as spurious FEEL-evaluation timeout incidents; resolved via Operate after confirming the cause.

Dead-letter handling: there is no dead-letter queue. Zeebe **incidents** are the dead-letter equivalent: the token stops at the failing task with its variables intact until the incident is resolved, after which the job is retried.

4.3 Monitoring and observability

Signal	Source
Process-level state, variables, incidents	Camunda Operate (<code>:8080/operate</code>)

Signal	Source
Task queues and SLA pressure	Portal /tasks (open count per role), Camunda Tasklist
Business-level case history	audit_log (portal /audit page; /case-trace developer skill merges audit_log with Camunda history and flags steps with no audit row)
Service logs	console output of the API and worker processes (each worker logs "<job-type> worker started, polling for jobs")
Engine health	GET :8080/v2/topology ("health":"healthy"), docker stats orchestration
Database health	pg_isready

There is no metrics backend, tracing, or alerting stack.

4.4 Audit trail as the integrity record

Every worker and user-task completion writes at least one audit_log row:

Actor type	Written by	Examples of action
system	Deterministic workers, the API	submitted, process-started, validated, routed, settlement_triggered, claimant_notified, case_closed, review_sla_escalated, triage_auto_confirmed, validation_exception_auto_rejected
ai	Gemini-backed workers (with model and promptVersion in detail)	extracted_evidence, detected_fraud_indicators, scored_risk, denial_letter_drafted
human	Capture workers on user-task completion; resubmission endpoint	triage_confirmed, rejected_at_triage, decision_recorded, signed_off, validation_exception_resolved, validation_exception_rejected, claimant_resubmission_submitted, claimant_resubmitted

Appendix A — Data Model

Table	Purpose	Key columns
policies	Policy book	policy_number (unique), carrier_id, insurance_type, policyholder_name/email/phone, status (active / lapsed / cancelled), effective_date, expiry_date, premium_amount ≥ 0, coverage_amount > 0
policy_dependents	Authorized claimants besides the policyholder	policy_id (FK, cascade), full_name, email, phone, relationship (spouse / child / other); unique (policy_id, email)
providers	Healthcare providers	npi (unique, 10 digits), tax_id, facility_name, facility_address
claims	One row per claim (system of record)	Identity & intake: id (uuid), carrier_id, insurance_type, policy_number, policy_id, claim_type (outpatient / inpatient / pharmacy / dental / maternity / other), claimant_name/email/phone, channel (portal / whatsapp), incident_date, incident_description, claim_amount, provider_id, diagnosis_code, procedure_code, service_date_from/to, total_billed_amount > 0, coordination_of_benefits, attestation_signed_at . Processing: status (submitted / validating / triage / in_review / awaiting_info / approved / denied), case_summary, risk_score, risk_reasoning, fraud_indicator_count, assigned_role, confirmed_role, triage_note, decision (approve / deny / moreInfo), denial_reason, info_requested_reason, denial_letter_text, settlement_id, process_instance_key, created_at, updated_at
claim_documents	Document references + AI extraction	claim_id (FK, cascade), file_url, document_type (unused), extracted_data (jsonb)
claim_fraud_indicators	AI-flagged indicators	claim_id, type, description, confidence
audit_log	Case history	claim_id, actor_type (system / ai / human), actor_id, action, detail (jsonb), created_at
users	Portal accounts	email (unique, case-insensitive), password_hash (bcrypt), role, reset_otp_*, token_version
whatsapp_sessions	WhatsApp conversation state	phone_number (unique), mode, collected_fields (jsonb), documents (jsonb), status
whatsapp_processed_messages	Webhook idempotency	message_id (PK), received_at

Table	Purpose	Key columns
schema_migrations	Applied migrations	version (PK), applied_at

Relationships: policies 1-* policy_dependents, policies 1-* claims, providers 1-* claims, claims 1-* claim_documents, claims 1-* claim_fraud_indicators, claims 1-* audit_log.

Appendix B — Process Variables Contract

Variables set at process start by createClaim() :

claimId, carrierId, insuranceType, policyNumber, claimType, claimAmount, policyholderName, coverageAmount, claimantName, claimantEmail, incidentDate, incidentDescription, diagnosisCode, procedureCode, serviceDateFrom, serviceDateTo, totalBilledAmount, coordinationOfBenefits, providerFacilityName, providerNpi, documents[{ name, url, contentType, dataUri }]

Variables added during the process:

Variable	Set by	Used by
validationPassed, policyId, duplicatePendingClaim, duplicateClaimId, authorizedClaimant	validate-claim	Validation gateway, exception form
daysSincePolicyEffective, claimantClaimCountLast12Months	validate-claim	DMN
caseSummary	extract-evidence	Fraud, risk, reviewers
fraudIndicatorCount	detect-fraud-indicators	Risk, DMN
riskScore, riskReasoning	score-risk	DMN, reviewers
assignedRole	DMN	Triage, auto-confirm
slaDeadline	validate-claim, capture-routing-decision, capture-triage-review, auto-confirm-triage, auto-escalate-review	Timer boundary events
resolutionAction, triageAction, confirmedRole, decision, denialReason, infoRequestedReason	User-task forms / portal task proxy	Gateways, capture workers
settlementId	trigger-settlement	notify-claimant
denialLetterText	draft-denial-letter	notify-claimant

Appendix C — Runtime Topology & Configuration

Process / container	Port(s)	Started by
claimflow-postgres (Postgres 16)	5432	docker-compose up -d (repo root)
claimflow-minio (MinIO)	9000 (S3), 9001 (console)	same
orchestration (Camunda 8.9.16)	26500 (gRPC), 8080 (Operate/Tasklist/REST), 9600 (management)	docker-compose up -d (camunda-docker/)
backend/api (Express)	4000	npm run dev
backend/workers (17 workers, one process)	— (outbound gRPC)	npm run dev
frontend/portal (Next.js)	3000	npm run dev
ngrok tunnel → API	public HTTPS static domain	npm run tunnel (backend/api)

Key configuration (environment):

Variable	Component	Purpose
DATABASE_URL	API, workers	Postgres connection
MINIO_ENDPOINT, MINIO_PORT, MINIO_ROOT_USER, MINIO_ROOT_PASSWORD, MINIO_BUCKET	API	Object storage
ZEEBE_GRPC_ADDRESS, CAMUNDA_AUTH_STRATEGY	API, workers	Engine connectivity (gRPC). The API's REST user-task client uses the SDK default address http://localhost:8080.
SESSION_SECRET, SESSION_TTL_HOURS	API	Bearer-token signing and lifetime (default 8 h)
CORS_ORIGIN	API	Allowed portal origin

Variable	Component	Purpose
GEMINI_API_KEY , GEMINI_MODEL , GEMINI_FALLBACK_MODELS	Workers	AI model and fallback chain
EMAIL_TRANSPORT_SELECTOR , GMAIL_USER , GMAIL_APP_PASSWORD , RESEND_API_KEY	API, workers	Email transport selection
WHATSAPP_PHONE_NUMBER_ID , WHATSAPP_ACCESS_TOKEN , WHATSAPP_APP_SECRET , WHATSAPP_WEBHOOK_VERIFY_TOKEN	API	WhatsApp channel
FRONTEND_URL	Workers	Links in claimant emails
NEXT_PUBLIC_API_BASE_URL	Portal	API base URL

Appendix D — Known Technical Limitations (as-built)

Area	Limitation
Intake durability	No transactional outbox: if Zeebe is down after the DB commit, the claim has no process instance and nothing re-drives it.
Error modeling	No BPMN error boundary events; all failures become incidents after retries.
Email routing	All real emails (claimant outcomes, reviewer notices, resubmission notice) go to one hardcoded test recipient, with the intended recipient in the subject.
Settlement	Mock provider only.
Duplicate detection	Any open claim on the same policy counts as a duplicate; no time window or similarity check.
Server-side file typing	Upload type restriction is client-side only; <code>claim_documents.document_type</code> is never populated.
Audit attribution	Human task completions are audited with <code>actor_id = "tasklist"</code> , not the reviewer's user id.
Message correlation	The "more info" loop is a user task completed via REST rather than a BPMN message correlation.
Multi-carrier	<code>carrier_id</code> is stored and used in policy matching, but tenant isolation is not enforced.
Observability	Console logs only; no metrics, tracing or alerting.
Housekeeping	No cleanup of <code>whatsapp_sessions</code> / <code>whatsapp_processed_messages</code> ; no data retention jobs.
Spec drift	<code>SPEC.md</code> describes Resend-first email (code prefers Gmail) and message-event correlation for "more info" (code uses a user task); <code>validate-claim</code> reads <code>incidentDate</code> from the DB rather than the process variable.